# 20 Agentic Design Patterns (Part 3)

A comprehensive guide to 20 agentic design patterns that separate pros from beginners, based on a Google engineer's 400-page book. Practical patterns you can use today with plain English explanations.

Back

## Navigation

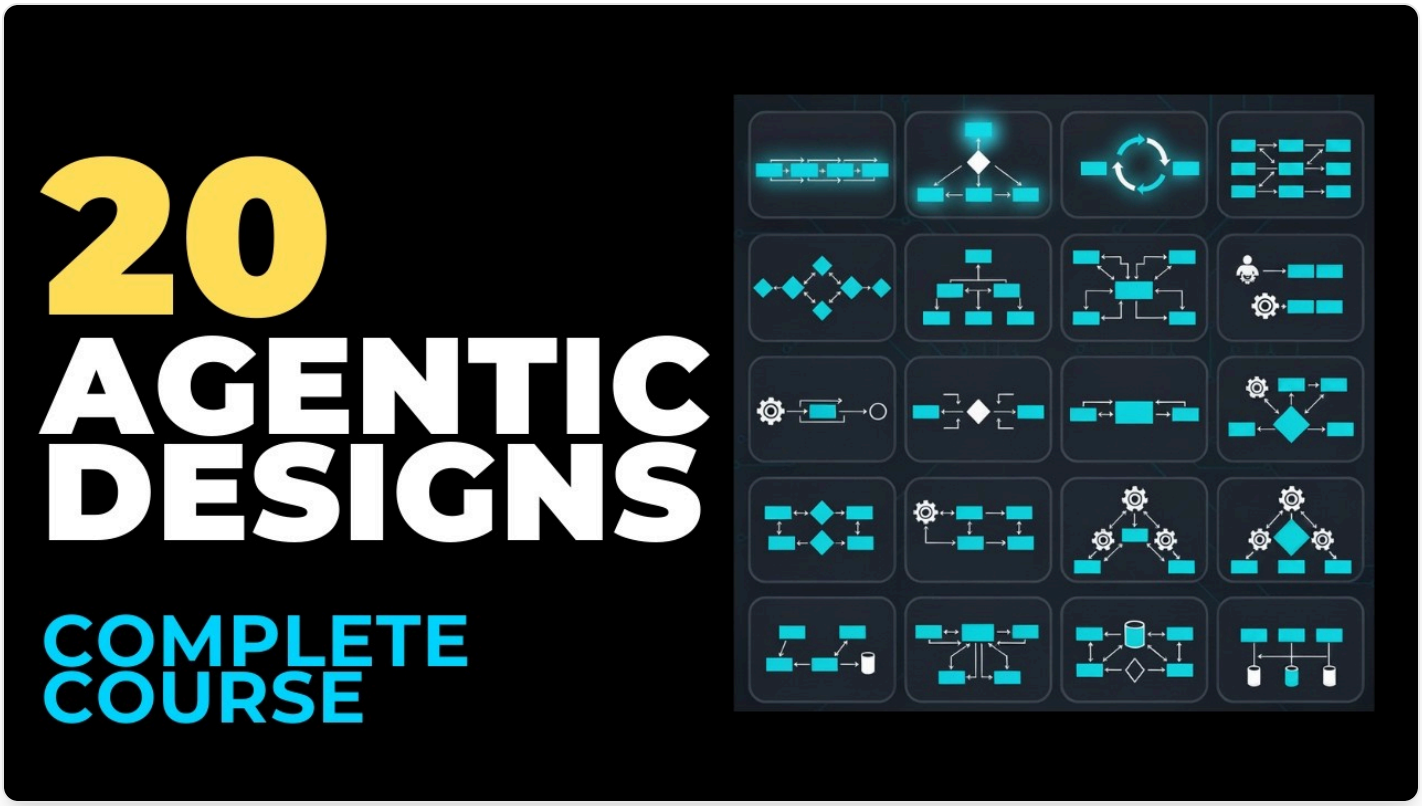
This guide is split into 4 parts for better performance:


- Part 1: Chapters 1-5 - Prompt Chaining, Routing, Parallelization, Reflection, Tool Use
- Part 2: Chapters 6-10 - Planning, Multi-Agent Collaboration, Memory Management, Learning and Adaptation, Goal Setting and Monitoring
- Part 3: Chapters 11-15 - Exception Handling and Recovery, Human in the Loop, Knowledge Retrieval (RAG), Inter-Agent Communication, Resource-Aware Optimization
- Part 4: Chapters 16-20 - Reasoning Techniques, Evaluation and Monitoring, Guardrails and Safety Patterns, Prioritization, Exploration and Discovery

Originally video talking about agentic systems.

You can help out the author who broke down the 400-page manual published by the Google engineer here.

Link not affiliated.




## Chapter 11: Exception Handling and Recovery

**TLDDR:** This is just the way that you catch errors in your agentic workflows. This is an agentic pattern to help catch issues in your other agentic patterns. So essentially what you're trying to do is you do something, you add safety checks, you make the call to these services or tools or both.

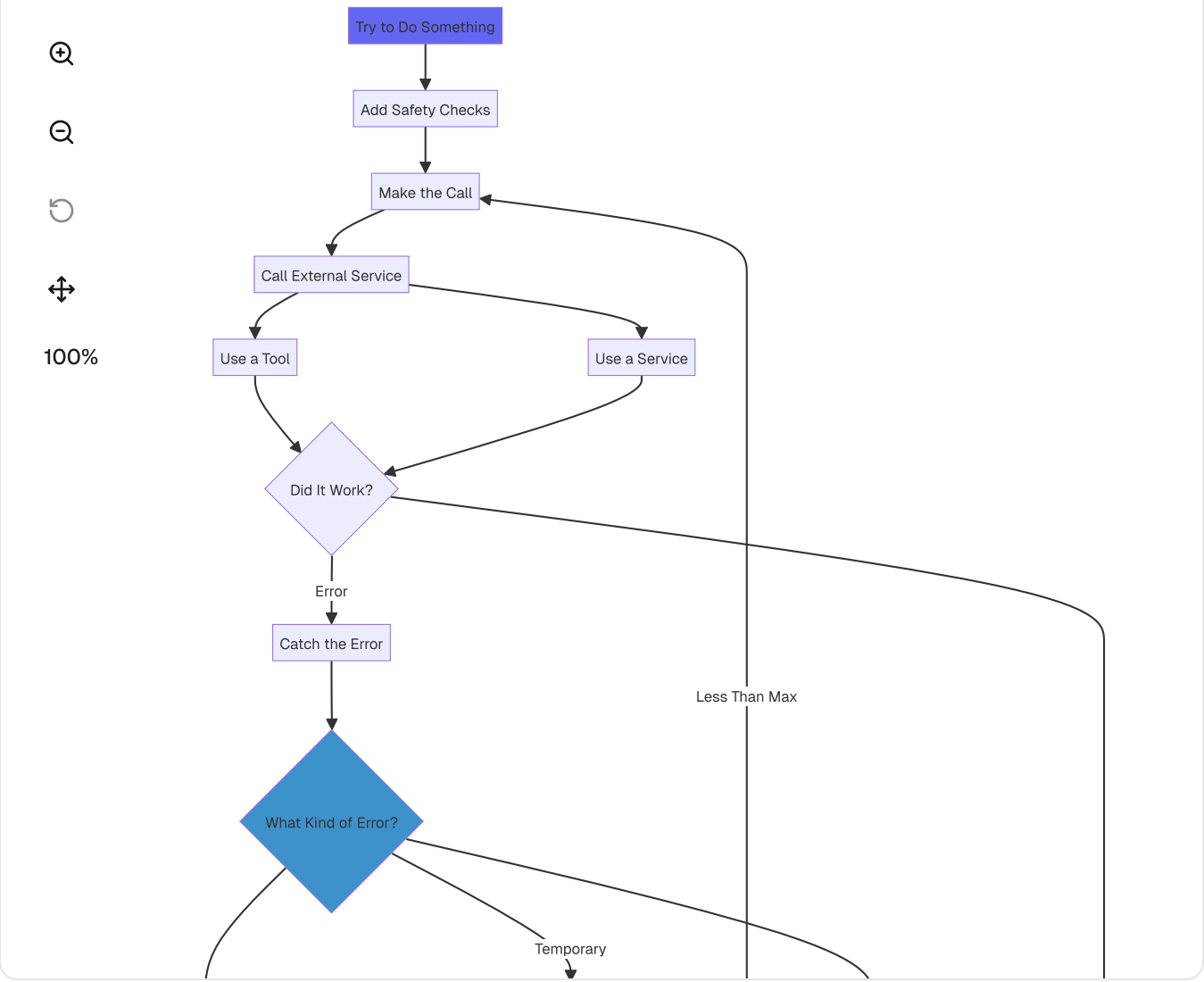
When to Use Where It Fits

## When to Use

- **Production environments**: Any system requiring high reliability
- **External dependencies**: When relying on APIs or services
- **Critical operations**: Tasks that must not fail completely
- **Unpredictable inputs**: Handling edge cases and anomalies
- **Network operations**: Managing connectivity issues
- **Resource constraints**: Dealing with limits and quotas

## How It Works

MermaidZoom MMD




How does exception handling work?

Add safety checks, make the call to services or tools. Catch errors, classify what kind of error it is

What kinds of errors?

What's exponential backoff?

Wait 1 minute, then 2 minutes, then 4 minutes. Cap it so you don't retry forever if it's actually permanent

What backup options?

Use simpler method, saved data, default answer, or get human help

What about critical errors?

Save current work, alert team, check if safe to continue. If not safe, emergency stop. Otherwise resume

Pros Cons

## Pros

- **Reliability**: System continues operating despite failures
- **Graceful degradation**: Provides partial functionality when full service unavailable
- **Self-healing**: Automatic recovery from transient issues
- **User experience**: Minimizes disruption to users
- **Debugging support**: Comprehensive error logging
- **Learning capability**: Improves handling over time
- **State preservation**: Can resume after interruptions

Payment Processing | Data Pipeline | Chatbot Service | Content Delivery | ML Pipeline | IoT Devices

**Payment Processing System**

- Retry failed transactions with backoff
- Fallback to alternative payment gateways
- Save transaction state for manual review
- Notify finance team of critical failures
- Automatic refund on persistent failures

---

## Chapter 12: Human in the Loop

**TLDDR:** Adding a human in the loop where there's low to high risk depending on the situation or most importantly edge cases. So you have some form of agent processing, you have a decision point and one of those decisions could be that a review is needed or you need to actually step in and intervene. A good actual tactical example is imagine you're using some form of agentic browser or agent mode in ChatGPT. At some point it will realize it needs you to step in to add your credentials to log in to your email to Upwork to whatever service it is.

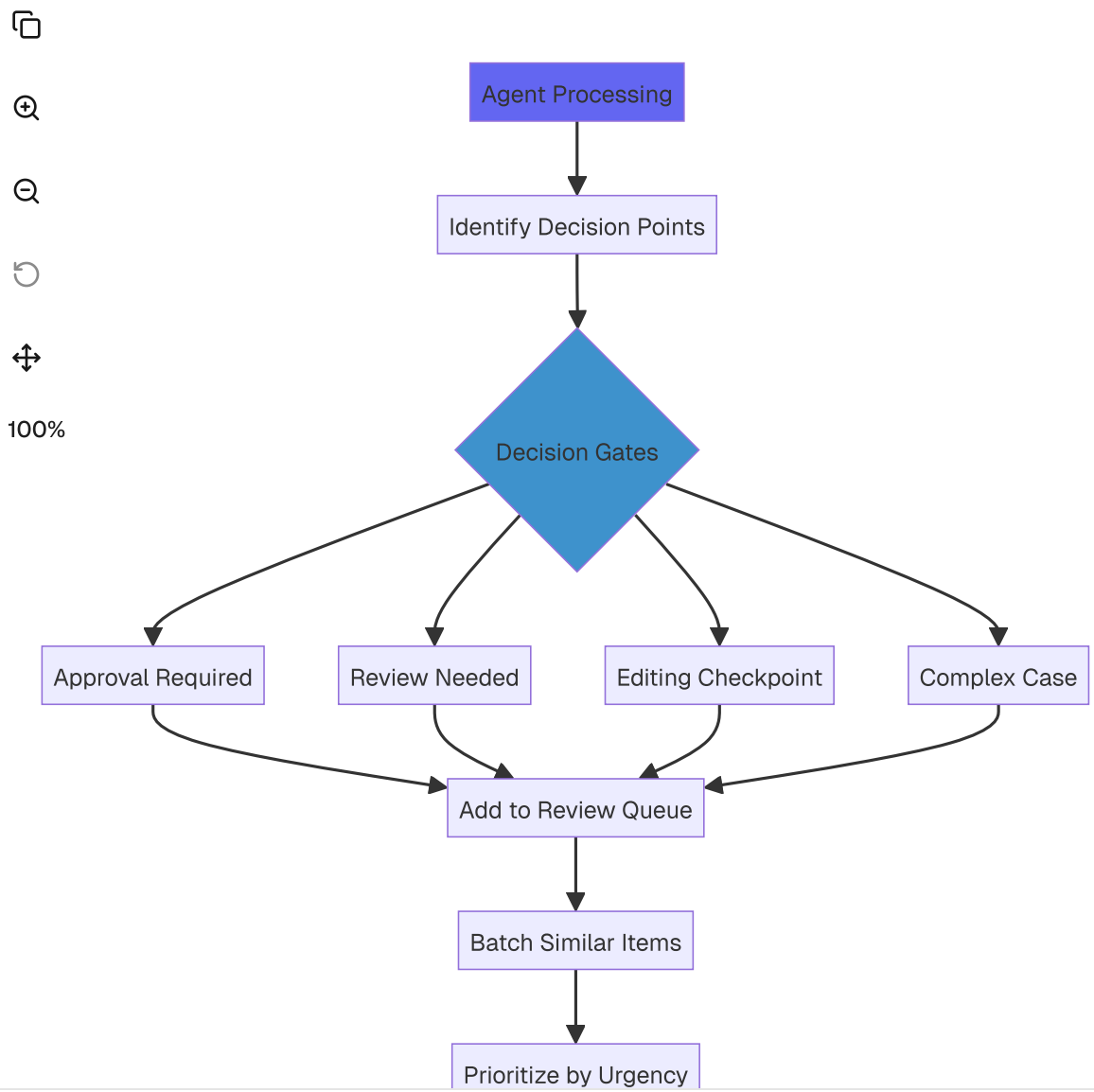
When to Use | Where It Fits

### When to Use

- **High-stakes decisions**: When errors have significant consequences
- **Regulatory compliance**: Required human oversight for legal reasons
- **Quality assurance**: Ensuring output meets standards
- **Edge cases**: Handling unusual or ambiguous situations
- **Training data generation**: Using human feedback to improve

## How It Works

MermaidZoom MMD




How does human in the loop work?

Agent identifies decision points - approval needed, review needed, editing checkpoint, complex case

Add to review queue, batch similar items, prioritize by urgency. Present in UI with full context, show differences, display SLA timer

What can humans do?

Approve, deny with reason, edit content, or take full manual control

How does it learn?

Capture rejection patterns, record edit changes, log takeover reasons. Update agent training to improve future decisions

What about human fatigue?

Monitor fatigue levels. If high, reduce human load by increasing automation. Track decision metrics and generate reports

Pros Cons

## Pros

- **Quality assurance**: Human judgment catches AI errors
- **Compliance**: Meets regulatory requirements
- **Learning source**: Human feedback improves system
- **Trust**: Users confident in human oversight
- **Flexibility**: Humans handle edge cases well
- **Accountability**: Clear responsibility chain

## Real-World Examples

Content Moderation | Loan Approval | Medical Imaging | Resume Screening | Translation QC | Autonomou

**Content Moderation Platform**

- AI flags potentially problematic content
- Human reviewers make final decisions
- Complex cases escalated to senior moderators
- Reviewer feedback trains AI models
- Fatigue monitoring and rotation schedules

---

# Chapter 13: Knowledge Retrieval (RAG)

**TLDDR:** Indexing documents by parsing, chunking, and creating searchable embeddings. Literally RAG. So it's like having a librarian and you want to categorize or index a series of information and systems. So this one is pretty straightforward where you have a user query. You have some sources that you've ingested. You've parsed those documents, categorized them, embedded them, which again means in plain English, you take words, you turn them into vectors, you store vectors into library. So when you ask a question, you try to match the vector of the question to the vectors in your library with the closest match.

When to Use | Where It Fits

## When to Use

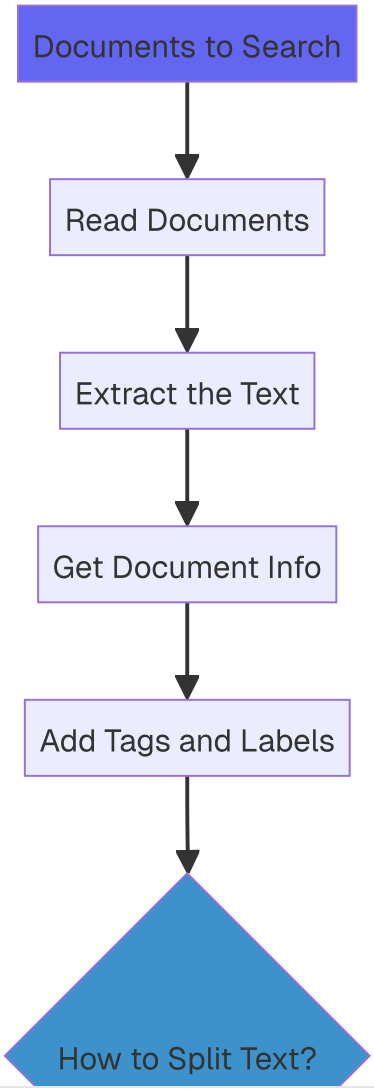
- **Dynamic knowledge needs**: Accessing up-to-date information
- **Large document collections**: Querying extensive knowledge bases

- **Citation requirements**: Providing verifiable references
- **Reducing hallucinations**: Ensuring factual responses

## How It Works

MermaidZoom MMD

100%




How does RAG work?

Parse documents, chunk them (fixed size, natural breaks, or context-aware), convert to embeddings (vectors), store in search database

What happens when user asks question?

Improve question, expand with related words, search database. Find matching chunks, filter irrelevant ones, rank by relevance

How do you rank?

Score each match, sort best to worst, pick top K matches (usually 5-10). More matches = more context but also more risk of hallucination

Then what?

Check if sources are good, use them to generate answer, add citations. If answer quality is bad, try different search with adjusted settings

How do you improve?

Track accuracy and coverage, measure success, optimize the system based on results

Pros Cons

## Pros

- **Accuracy**: Responses grounded in real sources
- **Verifiability**: Citations enable fact-checking

- **Domain expertise**: Specialized knowledge integration
- **Reduced hallucination**: Less fabrication of facts
- **Flexibility**: Easy to update knowledge base

### Real-World Examples

Enterprise Knowledge | Legal Research | Medical System | Academic Research | Technical Support | New

**Enterprise Knowledge Management**

- Index company policies and procedures
- Retrieve relevant HR guidelines
- Search technical documentation
- Access historical project data
- Provide sourced answers to employees

---

## Chapter 14: Inter-Agent Communication

**TLDDR:** Agents communicate through a structured messaging system with defined protocols. Message including IDs for tracking expiration times and security checks. So this is like an office email system with read receipts, security clearances and spam filters that prevent reply all disasters. So this is where you have language models talking to other language models. From a system perspective, this is where you could have multiple AI agents speak to one another and then you have to decide how they should communicate. Either they have one boss, one that manages all the other agents, which is sometimes really helpful to have because you have a single vector of failure that everything can report to. The next one is that everyone is equal, meaning everyone has a say at the table. It is a pure agentic democracy which sounds great but

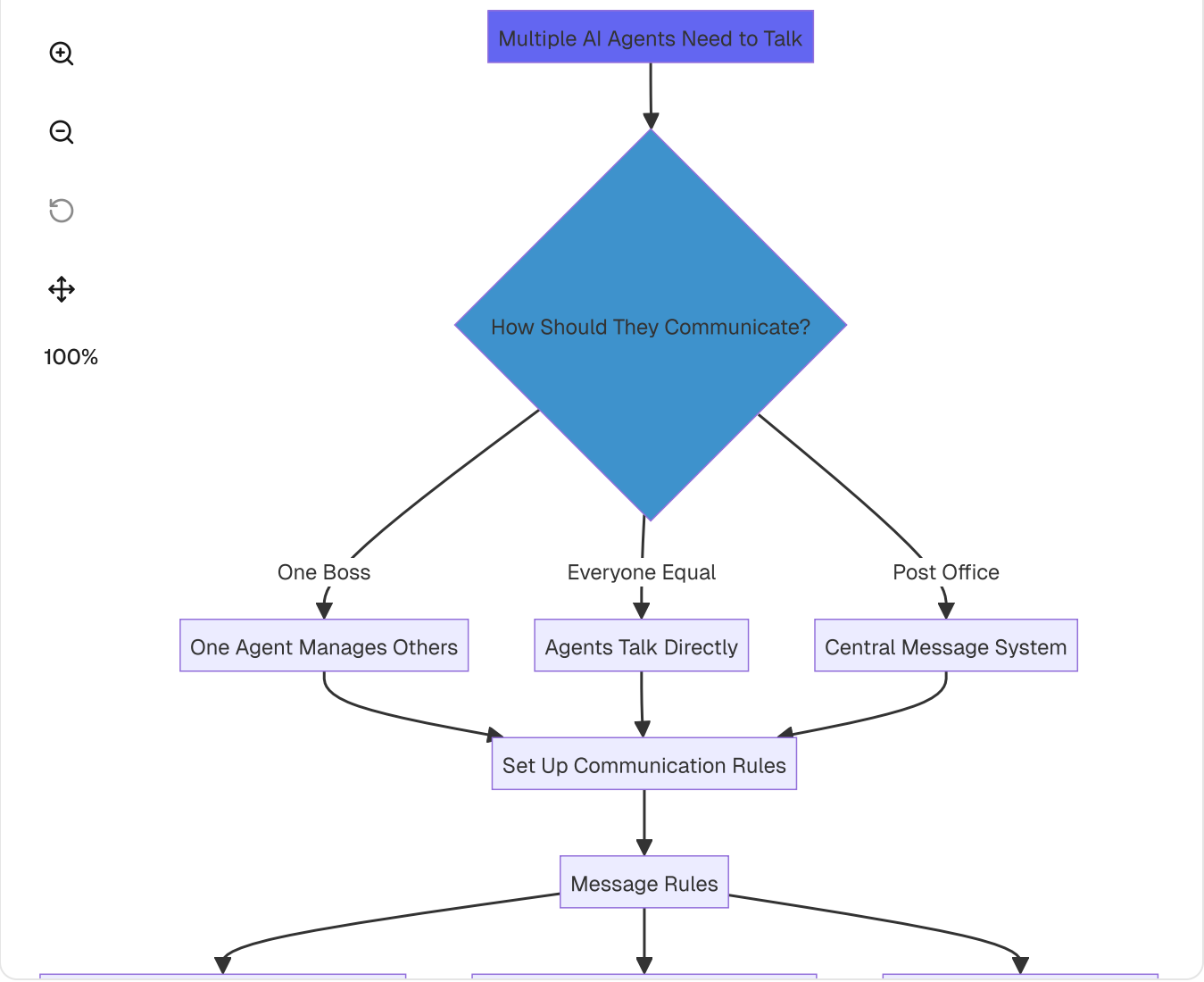
When to Use Where It Fits

## When to Use

- **Complex workflows**: Tasks requiring multiple specialized agents
- **Modular systems**: Building composable agent architectures
- **Distributed processing**: Agents running in different locations
- **Scalable architectures**: Systems that need to grow
- **Collaborative tasks**: Agents working together on problems
- **Service-oriented design**: Agents as microservices

## How It Works

MermaidZoom MMD




How do agents communicate?

Choose communication pattern - one boss managing others, everyone equal (direct), or central message system (mailbox)

What are message rules?

How do you control who can talk?

Verify agent identity, check permissions, allow communication. Send message, deliver to right agent

What if something goes wrong?

Monitor for endless loops, stuck agents, messages too long. Stop loops, unstick agents, cancel old messages, alert human if needed

Is this practical?

Makes a good YouTube video but not a really good production system. Very complex, lots of debugging. Enterprise level makes sense if you have tons of resources and engineers

Pros Cons

## Pros

- **Modularity**: Clear separation of agent responsibilities
- **Scalability**: Easy to add new agents to the system
- **Flexibility**: Different communication patterns available
- **Fault isolation**: Agent failures don't crash system
- **Reusability**: Agents can be reused in different workflows
- **Debugging support**: Message tracing aids troubleshooting
- **Parallel processing**: Agents can work simultaneously

## Real-World Examples

E-commerce Orders | News Production | Financial Analysis | Smart Manufacturing | Healthcare | Researc

**E-commerce Order Processing**

- Inventory Agent checks stock availability
- Pricing Agent calculates total costs
- Payment Agent processes transactions
- Shipping Agent arranges delivery
- Notification Agent updates customer
- Orchestrator coordinates entire flow

---

# Chapter 15: Resource-Aware Optimization

**TLDDR:** Analyzing a task complexity and then routing to appropriate resources. So simple tasks use cheap, fast models, but complex tasks use powerful but expensive models. Think of something like GPT-5 where there was a huge uproar because we lost all of our models. Then we got either quick thinking, kind of thinking, hard thinking or like professional thinking. Each one of those would route your request in ChatGPT to the model that it thought would be the best suited for that particular outcome. So the analogy here is a playful one where it's like choosing between walking, a bus or a taxi depending on the distance, the urgency or the budget.

When to Use | Where It Fits

## When to Use

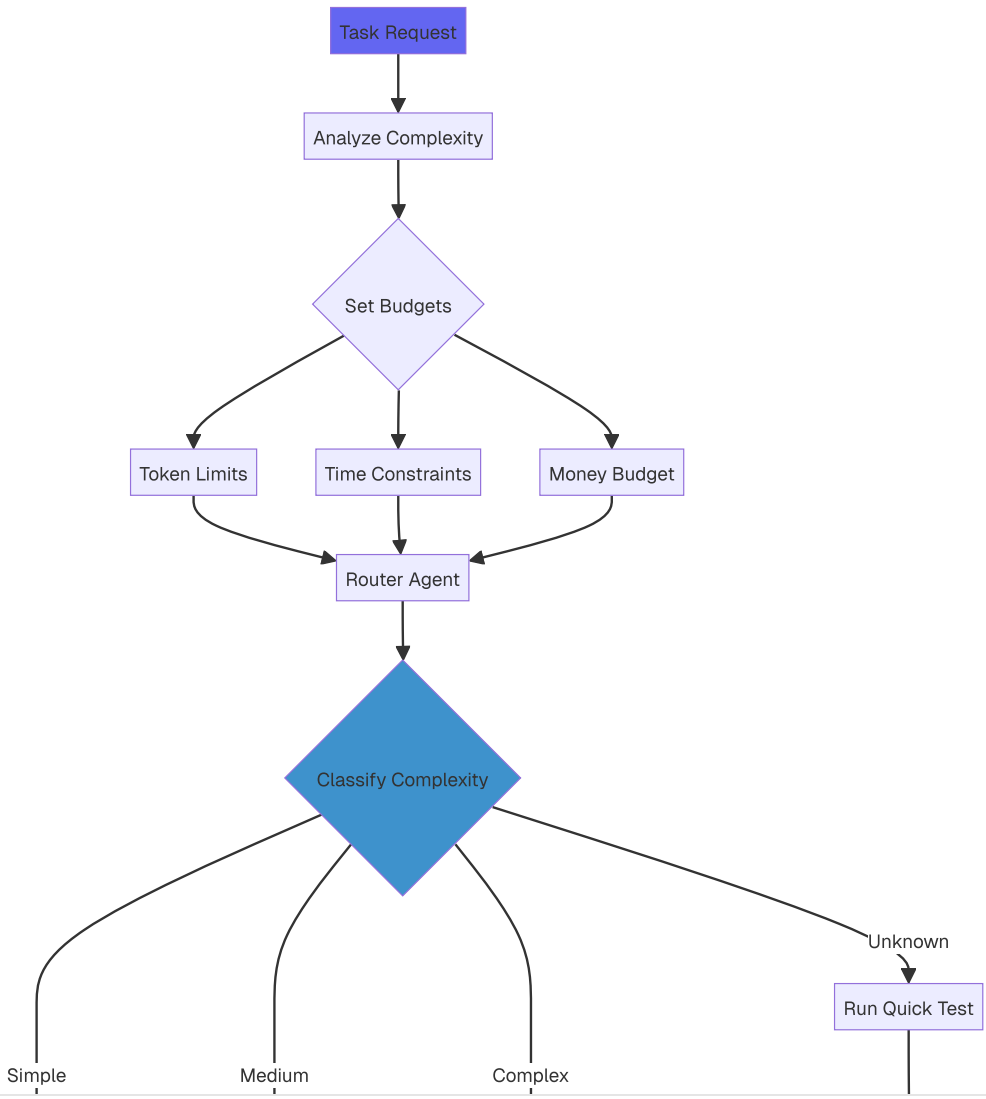
- **Cost-sensitive operations**: When managing API or compute costs
- **High-volume processing**: Optimizing large-scale operations

- **Performance requirements**: Balancing speed vs cost
- **Multi-tenant systems**: Fair resource allocation across users

## How It Works

MermaidZoom MMD

100%




How does resource-aware optimization work?

Analyze task complexity, set budgets (token limits, time, cost). Router agent classifies complexity

How does it route?

Built by neldivad. Copyright © 2024 - 2026 Nelworks Inc.

Simple → small model, medium → standard model, complex → advanced model. Unknown → run quick test, check confidence

What if you go over budget?

Monitor token count, response time, API costs. If over limits, optimize: prune context, use cached results, or switch to cheaper model

How do you improve?

Measure quality vs cost, calculate delta, tune thresholds, update router logic. That was what all the uproar around GPT-5 was - routing as many requests as possible to cheapest model while still charging you $20/month

Pros Cons

## Pros

- **Cost reduction**: Significant savings on API and compute costs
- **Performance optimization**: Right-sized resources for each task
- **Scalability**: Efficient resource use enables growth
- **Flexibility**: Dynamic adjustment to workload changes
- **Budget control**: Predictable operational costs
- **Quality preservation**: Maintains output quality where needed

## Real-World Examples

Customer Support | Content Generation | Code Assistant | Translation Platform | Data Analysis | Educati

**Customer Support Platform**

- Simple FAQs use lightweight models
- Complex issues use advanced models
- Cache common question responses
- Prioritize premium customers
- Track cost per ticket resolution